TDS Prep Session 4 - Notes & Key Takeaways

Date: 2026/01/18

Speakers: Carlton D'Silva, Sujal Raj Pradhan

Topic: Version Control (Git), Project Structure, and Security Basics

1. Course Context & Workflow

- **The TDS Approach:** The course focuses specifically on "Tools in Data Science," bridging the gap between theoretical data analysis and practical application engineering.
- **Workflow:** The curriculum follows a specific, industry-relevant pipeline: **Sourcing -> Deployment -> Tools**.
  - *Note:* In a shift from previous iterations, **Deployment** will be taught *before* **Sourcing** this term. The rationale is to understand the "destination" (where the app lives) before building the "source" (data pipelines).
  - Concepts like HTML structure and HTTP requests (covered briefly) are foundational precursors to Sourcing, necessary for web scraping and API interaction.
- **Expectations & Engineering Mindset:** The course involves significantly less handholding than foundation courses.
  - Instructors provide "kickstarter" demos and introductions to tools.
  - **The "Figure It Out" Philosophy:** Students are expected to rely on official documentation, Stack Overflow, and AI tools (like ChatGPT) to resolve specific implementation details. This mimics a real-world software engineering environment where navigating documentation is a core skill.

2. Version Control (Git & GitHub)

Why Use Version Control?

- **The Problem:** Without version control, collaboration becomes a manual nightmare. Sharing files via Google Drive or WhatsApp leads to **"Version Chaos"** (e.g., `analysis_v1.py`, `analysis_v2_FINAL.py`, `analysis_v2_FINAL_real.py`). This method is error-prone, makes merging work impossible, and has no audit trail.
- **The Solution:** Git is the industry standard for source code management. It allows multiple developers to work on the same codebase simultaneously (branching), tracks every single

line of code changed (history), and enables safe experimentation with the ability to revert to previous states (rollbacks).

Core Concepts & Terminology

- **Git vs. GitHub:**
  - **Git:** The distributed version control system installed **locally** on your OS. It handles the logic of tracking changes and history on your machine.
  - **GitHub:** The cloud-based hosting service (**Remote**). It acts as the central hub where the Git repositories are stored, shared, and backed up online.
- **The "Kitchen" Analogy (The Three Stages of Git):**
  - **1. Coding (Working Directory):** The messy kitchen counter where you chop ingredients. This is where you write and modify code. It is untracked and volatile.
  - **2. Staging Area (`git add`):** Plating the food. You select specific changes (ingredients) that are ready to be served.
    - *Why stage?* It allows you to group related changes together. You might have edited 10 files, but you only want to commit the 3 files related to "User Login" first. Staging lets you curate the snapshot.
  - **3. Commit (`git commit`):** Serving the dish. This creates a permanent, immutable snapshot of the staged changes in the project's history.
- **Commit Hash vs. Checksum:**
  - **Commit SHA:** A unique 40-character string generated by Git to identify a specific commit (e.g., `a1b2c3d...`). It tracks the *timeline*.
  - **Checksum (SHA256):** A cryptographic hash of a file's *content*. This is often used in TDS assignments to verify that a file has not been tampered with. *Do not confuse the history ID (Commit SHA) with the content ID (Checksum).*

Key Commands Covered

1. `git init` : Turns the current folder into a Git repository.
2. `git add .` (Stage all) or `git add <file>` (Stage specific): Moves files from the Working Directory to the Staging Area.
3. `git commit -m "message"` : Saves the snapshot. *Tip: Write descriptive messages (e.g., "Fix login bug" instead of "Update").*
4. `git remote add origin <url>` : Links your local folder to a specific repository on GitHub, creating a bridge for upload/download.

5. `git push origin master` : Uploads your local commits to the remote server, syncing your work with the team.
6. `git status` : The most useful command. It tells you exactly where your files are: untracked, staged, or modified.

Tools Recommended

- **VS Code:** Excellent for beginners. The "Source Control" tab provides a graphical interface to stage, commit, and sync without memorizing every CLI command.
- **WebStorm:** A powerful IDE recommended for advanced visualization of Git trees and resolving complex merge conflicts.
- **GitLens:** A VS Code extension that adds "blame" annotations (showing who wrote which line of code and when) directly in the editor.

3. Project Structure & Organization

Beginners often "dump" all files (scripts, data, images) into a single root folder. As projects grow, this causes pathing errors and makes deployment impossible. A standardized structure is mandatory for professional development.

Recommended Directory Structure

- `src/` **(Source):** Contains the actual application code.
  - Includes modules like `main.py` , `utils.py` , or `config.py` .
  - *Benefit:* Keeps logic separate from data and artifacts.
- `data/` **:**
  - Often split into `data/raw` (read-only original data) and `data/processed` (cleaned data).
  - *Note:* Large datasets should generally not be committed to Git.
- `logs/` **:** Dedicated folder for runtime logs. Keeping these separate prevents them from cluttering the source folder or being mistaken for code.
- `outputs/` **:** The destination for any file generated by the code (graphs, reports, CSV predictions). This ensures that running the code doesn't overwrite your source data.

4. Logging vs. Outputs vs. Errors

In a production environment, simply printing to the console is insufficient. You must distinguish between the types of information your application produces:

Type	Definition	Best Practice & Example
Logs	Telemetry regarding the *internal* execution flow. Used by developers to debug or monitor health.	Use the `logging` library. *Ex:* `INFO: User 42 connected from IP x.x.x.x`
Outputs	The *deliverable* value. This is the artifact the user or downstream system actually needs.	Save to `outputs/` folder. *Ex:* `quarterly_report.pdf`, `forecast.csv`.
Errors	Critical signals that the application failed to perform a task.	specific Exception handling (`try/except`). *Ex:* `404 Not Found`, `ZeroDivisionError`.

- **Why not `print()`?** `print()` statements are ephemeral and hard to filter. Python's `logging` module allows you to set severity levels (DEBUG, INFO, WARNING, ERROR), timestamp events automatically, and write to persistent files in the `logs/` directory.

5. Security Basics

Security is as much about discipline/habits as it is about tools.

- **API Keys & Secrets:**
  - **The Rule:** NEVER hardcode secrets (OpenAI keys, Hugging Face tokens, Database passwords) directly into your Python scripts.
  - **The Risk:** If you push a hardcoded key to a public GitHub repo, scrapers will find it in seconds and abuse your quota/billing.
- **Environment Variables:**
  - The standard industry practice is to inject secrets as environment variables.
  - **Implementation:** Create a `.env` file (a simple text file with `KEY=VALUE` pairs). Use the `python-dotenv` library to load them into your code:

```
import os
from dotenv import load_dotenv
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY") # Safe!
```

- **The Critical Role of `.gitignore`:**
  - This is a text file that tells Git which files or folders to **intentionally ignore**.
  - **Must Ignore List:**
    - `.env` (Contains secrets!)
    - `__pycache__/` (Python compiled bytecode)
    - `venv/` or `.env/` (Local virtual environment folders)
    - `data/` (If datasets are large or private)
    - `outputs/` (Generated files shouldn't usually be version controlled)
- **Remediation (What if I leak a key?):**
  - Git history is immutable. Simply deleting the line and pushing a new commit **does not**